

UMBRA V3

Dark Web De-Anonymization Intelligence Platform

COMPLETE DEPLOYMENT & OPERATIONS GUIDE

National Investigation Agency (NIA) | Law Enforcement Use Only

India CERT-In | Cyber Crime Units | Intelligence Agencies

Classification	LAW ENFORCEMENT SENSITIVE — DO NOT DISTRIBUTE
Version	3.0 — Final Production Release
Platform	Kali Linux 2023+ / Ubuntu 22.04 LTS / Debian 12
Architecture	Python FastAPI Backend + React D3.js Frontend
Engines	12 Intelligence Modules, 28 PII Patterns
Live APIs	Tor, ip-api.com, Blockchair, WalletExplorer, GitHub, Reddit, OpenPGP, crt.sh, Shodan
AI Analysis	Claude Opus (claude-opus-4-5) — NIA OSINT Brief
Target Problem	De-anonymizing .onion operators: real IP, VPN IP, identity, location, infrastructure
Prepared for	National Investigation Agency, India / Field Demonstration

1. WHAT UMBRA V3 DOES — Mission Overview

UMBRA V3 is a real, operationally-deployed dark web intelligence platform. It connects to the Tor network, fetches .onion sites through a live Tor circuit, and automatically runs 12 intelligence engines to produce actionable de-anonymization intelligence for law enforcement analysts.

⚠ This is NOT a simulation. UMBRA connects to real Tor hidden services and queries real live APIs. All intelligence output is derived from actual network traffic and live internet databases.

Problem Statement

Operators of illegal .onion sites (drug markets, weapons, data leaks, cybercrime services, violence-for-hire) use Tor + VPN layers to hide their real identity and physical location. UMBRA systematically extracts every available indicator from the .onion site itself — through technical errors, digital artifacts, reused identities, and infrastructure correlations — to build a de-anonymization intelligence package.

12-Module Intelligence Pipeline

#	Engine	What It Does	APIs / Data Sources
1	Tor Fetch	Real SOCKS5h connection through live Tor circuit. Full HTML + headers.	127.0.0.1:9050 (Tor daemon)
2	PII Extraction	28 regex patterns: email, BTC/ETH/XMR/LTC/ZEC, IPs, Telegram, PGP, GA IDs, Stripe, S3, SimpleX, Jabber, Session, Wickr, I2P, Briar.	Local regex engine
3	IP Geolocation + VPN	All IPs from page + headers → country, city, lat/lon, ISP, ASN, VPN/proxy/datacenter flags. X-Forwarded-For = real IP leak.	ip-api.com (free, 45/min)
4	HTTP Header Analysis	X-Forwarded-For real IP leak, server fingerprint, timezone, Via chain, ETag, Last-Modified, CDN detection.	Parsed from Tor response
5	Stylometry	Burrows Delta suite: TTR, Hapax, Yule K, Honore R, function word profile (60 words), char trigrams, language detection (Hindi/Russian/Spanish/British/American).	Local NLP
6	Page Intelligence	BeautifulSoup: platform fingerprint, market category detection, clearnet links, onion links, external scripts, meta tags.	Local HTML parser
7	Certificate Transparency	Live crt.sh query for all clearnet domains found. Full cert history, SAN domains, first-seen date.	crt.sh JSON API
8	Blockchain Forensics	BTC balance, tx count, first/last activity via Blockchair. WalletExplorer exchange label for KYC legal process.	Blockchair API, WalletExplorer API
9	Identity Correlation	GitHub user/search API, Reddit about.json, OpenPGP keyserver search for all usernames/emails/PGP keys. Auto-generated Google dorks.	GitHub API, Reddit API, keys.openpgp.org
10	Infrastructure Fingerprint	Favicon MurmurHash3 (Shodan-compatible), GA/GA4/FB Pixel/Stripe/S3 analytics IDs, JS library versions, CDN detection.	mmh3 library + regex
11	Attribution Graph	D3.js force-directed graph: all artifacts as typed nodes, evidence chains as directed edges with confidence ratings.	networkx-style in-memory
12	Claude OSINT Brief	All 11 module outputs → Claude Opus: NIA operator profile, top 5 leads, opsec failures, copy-paste queries, legal recommendations.	Anthropic API (claude-opus-4-5)

Intelligence Output Per Scan

- Full IP geolocation with coordinates, ISP, ASN, VPN/proxy detection, Google Maps link
- All PII artifacts (emails, crypto addresses, phone numbers, usernames, API keys)
- Cross-platform identity correlation (GitHub profile, Reddit account, PGP key with real email)
- Infrastructure fingerprint (favicon Shodan hash, analytics IDs, JS fingerprint)
- D3.js interactive attribution graph: all artifacts as nodes, evidence chains as edges
- Auto-generated Google dorks and Shodan queries per artifact
- NIA-format Claude OSINT brief: operator profile, top 5 leads, legal recommendations

2. PREREQUISITES — What You Need

2.1 Hardware Requirements

CPU	Any modern x64 processor (2+ cores recommended)
RAM	4 GB minimum, 8 GB recommended
Storage	10 GB free space
Network	Active internet connection — Tor + API calls require internet
Display	Browser for frontend UI (Firefox or Chromium recommended)

2.2 Operating System

- Kali Linux 2023+ (recommended — Tor pre-installed, security tools available)
- Ubuntu 22.04 LTS / Debian 12 (fully supported)
- macOS 13+ (supported with Homebrew)
- Windows → use WSL2 (Windows Subsystem for Linux) running Ubuntu

2.3 Accounts / Keys Required

- Anthropic API Key — for Claude OSINT brief. Get at: console.anthropic.com (free tier works). Optional — platform runs without it but no AI brief.
- No other accounts required. All other APIs (ip-api.com, [Blockchair](https://blockchair.com), [WalletExplorer](https://walletexplorer.com), [GitHub](https://github.com), [Reddit](https://reddit.com), [OpenPGP](https://crt.sh), crt.sh) are free and unauthenticated.

⚠️ GitHub API unauthenticated: 60 requests/hour. For high-volume use, create a GitHub Personal Access Token (free) at github.com/settings/tokens

3. COMPLETE INSTALLATION — Kali Linux (Step by Step)

⚠ Follow steps in exact order. Do not skip any step. Each step must complete without errors before proceeding.

STEP 01 Open Terminal

Boot Kali Linux. Log in. Right-click desktop → 'Open Terminal Here', OR press Ctrl+Alt+T.

Verify you have internet access:

```
ping -c 3 google.com
```

***i** You should see '3 packets transmitted, 3 received'. If not — fix your network before continuing.*

STEP 02 Update System Packages

Always update before installing new software:

```
sudo apt update && sudo apt upgrade -y
```

***i** This may take 2–10 minutes on first run. This is normal.*

STEP 03 Install Tor

Kali Linux includes Tor. Verify it is installed and configured:

```
sudo apt install -y tor
```

Configure Tor to accept SOCKS connections and enable ControlPort (needed for new circuit requests):

```
sudo nano /etc/tor/torrc
```

Find or add these lines (uncomment by removing the # at start):

```
SocksPort 9050
ControlPort 9051
HashedControlPassword
16:872860B76453A77D60CA2BB8C1A7042072093276A3D701AD684053EC4C
```

The hashed password above corresponds to password: umbra2024. Save the file: Ctrl+O, Enter, Ctrl+X

Start Tor and enable auto-start:

```
sudo systemctl start tor
sudo systemctl enable tor
sudo systemctl status tor
```

***i** Status should show: 'active (running)' with green dot. If not, run: `sudo journalctl -u tor -n 20`*

STEP 04 Verify Tor Is Working

Confirm Tor is routing traffic properly:

```
curl --socks5-hostname 127.0.0.1:9050
http://check.torproject.org/api/ip
```

Expected response:

```
{"IsTor":true,"IP":"x.x.x.x"}
```

⚠ If you see IsTor:false — Tor is not working. Restart with: `sudo systemctl restart tor` && sleep 15, then retry.

STEP 05 Install Python 3 and pip

Kali includes Python 3. Verify:

```
python3 --version
pip3 --version
```

If pip3 is missing:

```
sudo apt install -y python3-pip python3-venv
```

STEP 06 Create Project Directory

```
mkdir -p ~/umbra && cd ~/umbra
```

Copy `umbra_v3_backend.py` to this directory:

```
# If you have it on a USB drive or download location:
cp /path/to/umbra_v3_backend.py ~/umbra/
# Verify it is there:
ls -la ~/umbra/
```

STEP 07 Create Python Virtual Environment

Using a virtual environment keeps dependencies isolated:

```
cd ~/umbra
python3 -m venv venv
source venv/bin/activate
```

Your prompt should now show '(venv)' at the start. This means the environment is active.

i You must run 'source venv/bin/activate' every time you open a new terminal to use UMBRA.

STEP 08 Install All Python Dependencies

Install all required packages (copy this entire block, paste as one command):

```
pip install fastapi uvicorn "requests[socks]" PySocks stem anthropic
beautifulsoup4 lxml pydantic mmh3
```

Verify each critical package installed:

```
python3 -c "import fastapi, uvicorn, requests, socks, stem, anthropic,
bs4, lxml, mmh3; print('ALL OK')"
```

i You should see: ALL OK. If any import fails, install that package individually with: `pip install <package-name>`

If lxml fails to build (missing system libraries):

```
sudo apt install -y libxml2-dev libxslt-dev python3-dev
pip install lxml --force-reinstall
```

STEP 09 Set Anthropic API Key

For Claude AI OSINT brief, set your API key as an environment variable:

```
export ANTHROPIC_API_KEY="sk-ant-api03-YOUR-KEY-HERE"
```

To make this permanent (loads every time you open terminal):

```
echo 'export ANTHROPIC_API_KEY="sk-ant-api03-YOUR-KEY-HERE"' >>
~/.bashrc
source ~/.bashrc
```

***i** You can also paste your API key directly into the UMBRA frontend UI — no need to set environment variable.*

⚠ Never commit your API key to git or share it. The key is only used locally on your machine.

STEP 10 Start the UMBRA Backend

Make sure you are in the umbra directory with venv active:

```
cd ~/umbra
source venv/bin/activate
python3 umbra_v3_backend.py
```

Expected startup output:

```
=====
UMBRA V3 — Dark Web Intelligence Platform
NIA / Law Enforcement Use Only
=====
INFO:      Started server process
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

***i** Keep this terminal window open. The backend must stay running while you use UMBRA. Do not close it.*

⚠ If port 8000 is already in use: kill \$(lsof -ti:8000) then retry.

4. FRONTEND SETUP — Two Options

Option A: Claude.ai Artifacts (Easiest — No Setup)

This is the recommended option for demonstrations. No Node.js or build tools needed.

1. Open Claude.ai in Firefox or Chromium
2. Create a new conversation
3. Upload umbra_v3_frontend.jsx
4. Type: 'Render this React artifact'
5. Claude will display the full UMBRA UI directly in the browser

⚠ The frontend makes calls to `http://localhost:8000`. Claude.ai artifacts run in the browser and can reach localhost. This works correctly.

Option B: Local React Development Server

For a standalone deployment not dependent on Claude.ai:

STEP F1 Install Node.js

```
sudo apt install -y nodejs npm  
node --version    # Should be v16+
```

STEP F2 Create React App

```
cd ~/umbra  
npx create-react-app umbra-ui  
cd umbra-ui
```

STEP F3 Install D3

```
npm install d3
```

STEP F4 Replace App

```
cp ../umbra_v3_frontend.jsx src/App.jsx  
npm start
```

Frontend opens at `http://localhost:3000`

5. PRE-OPERATION VERIFICATION CHECKLIST

Run through this checklist before every session — especially before a live demonstration.

5.1 Automated Verification Script

Create and run this one-command verification:

```
cat > ~/umbra/verify.sh << 'SCRIPT'
#!/bin/bash
echo '=== UMBRA V3 Pre-Flight Check ==='
# 1. Tor service
systemctl is-active tor > /dev/null && echo '[OK] Tor service running'
|| echo '[FAIL] Tor not running - run: sudo systemctl start tor'
# 2. Tor SOCKS port
nc -zv 127.0.0.1 9050 2>&1 | grep -q succeeded && echo '[OK] Tor SOCKS5
port 9050 open' || echo '[FAIL] Port 9050 closed'
# 3. Tor exit verification
TOR_IP=$(curl -s --socks5-hostname 127.0.0.1:9050
http://check.torproject.org/api/ip 2>/dev/null)
echo $TOR_IP | grep -q '"IsTor":true' && echo "[OK] Tor exit confirmed:
$TOR_IP" || echo '[FAIL] Not routing through Tor'
# 4. Backend health
curl -s http://localhost:8000/ | grep -q 'UMBRA' && echo '[OK] Backend
running at localhost:8000' || echo '[FAIL] Backend not running - start
umbra_v3_backend.py'
# 5. Backend Tor integration
curl -s http://localhost:8000/api/tor/check | grep -q '"is_tor":true'
&& echo '[OK] Backend confirms Tor connection' || echo '[WARN] Backend
cannot confirm Tor'
# 6. Python packages
source ~/umbra/venv/bin/activate && python3 -c 'import
fastapi,anthropic,mmh3,bs4; print("[OK] All Python packages present")'
|| echo '[FAIL] Missing packages'
# 7. API key
[ -n "$ANTHROPIC_API_KEY" ] && echo '[OK] Anthropic API key set' ||
echo '[WARN] No API key - AI brief disabled'
echo '=== Check complete ==='
SCRIPT
chmod +x ~/umbra/verify.sh && ~/umbra/verify.sh
```

5.2 Manual Verification

- ✓ Tor service: `sudo systemctl status tor` → shows 'active (running)'
- ✓ Backend running: `curl http://localhost:8000/` → returns JSON with 'UMBRA V3'
- ✓ Tor check: `curl http://localhost:8000/api/tor/check` → 'is_tor':true
- ✓ Frontend loads in browser and shows UMBRA V3 interface

✓ Click 'CHECK TOR' button in UI → shows green 'TOR ACTIVE' with exit IP

6. RUNNING AN ANALYSIS — Full Walkthrough

6.1 Starting a Session

STEP 1 Open Terminal → Activate Environment → Start Backend

```
cd ~/umbra
source venv/bin/activate
export ANTHROPIC_API_KEY="sk-ant-api03-YOUR-KEY" # if not in .bashrc
python3 umbra_v3_backend.py
```

Leave this terminal open. Open a second terminal for any other commands.

STEP 2 Open Frontend in Browser

Navigate to Claude.ai and render umbra_v3_frontend.jsx, OR go to <http://localhost:3000> if using React dev server.

STEP 3 Verify Tor Connection

Click CHECK TOR button in the top-right of the UI. Confirm green 'TOR ACTIVE' status with a non-local exit IP.

STEP 4 Enter Target

In the SCAN tab:

- Paste the .onion URL in the target field (include <http://> prefix)
- Paste your Anthropic API key in the key field (or leave blank if set as env var)
- Press Enter or click the ANALYZE button

STEP 5 Wait for Analysis

UMBRA runs 12 modules sequentially. Typical timing:

Tor Fetch	5-40 seconds (depends on site responsiveness)
PII + Headers + Stylo	< 2 seconds (local computation)
IP Geolocation	1-4 seconds per IP (ip-api.com)
Certificate Transparency	2-5 seconds (crt.sh)
Blockchain	3-8 seconds (Blockchair)
Identity Correlation	5-15 seconds (GitHub + Reddit + PGP APIs)
Infra Fingerprinting	2-5 seconds (favicon download + mmh3)
Claude OSINT Brief	10-25 seconds (Anthropic API)
TOTAL	30-100 seconds typical

STEP 6 Review Results Across Tabs

- RESULTS — AI brief, PII artifacts, headers, stylometry, blockchain
- IP INTELLIGENCE — Full geolocation, VPN detection, Maps links, Shodan/GreyNoise/AbuseIPDB
- ATTRIBUTION GRAPH — Interactive D3 graph, drag/zoom/hover
- IDENTITY CORRELATION — GitHub/Reddit profiles, PGP keys, Google dorks
- INFRASTRUCTURE — Analytics IDs, favicon hash, JS fingerprint, CDN

- LOG — Full operation log for each step

7. IP & LOCATION INTELLIGENCE — Deep Dive

This is the most critical capability for physical attribution. UMBRA automatically extracts and geolocates all IPs through multiple channels.

7.1 IP Sources (Priority Order)

⚠ X-Forwarded-For header is the highest-priority finding. A hidden service running as a reverse proxy leaks the operator's real IP in this header. This alone can provide physical location.

Source 1 — X-Forwarded-For	HTTP header leak. Hidden service misconfigured as reverse proxy. Leftmost IP = operator real IP.
Source 2 — Via / Forwarded	HTTP proxy chain headers. May expose upstream server or VPN endpoint.
Source 3 — Page Source IPs	IPs embedded in HTML/JavaScript/comments. Often admin panel, API endpoint, or CDN IP.
Source 4 — Server header	Software version — enables CVE lookup and targeted exploit research.
Source 5 — Set-Cookie domain	Cookie set for a clearnet domain may reveal hosting infrastructure.

7.2 Geolocation Data Returned Per IP

- Country, Country Code, Region/State, City, Postal Code
- Latitude and Longitude coordinates ($\pm \sim 10\text{km}$ accuracy for residential)
- Timezone (corroborates operator active hours from server Date header)
- ISP name + Organization + ASN number + ASN name
- VPN/Proxy flag — known proxy/VPN exit nodes (ip-api.com proxy database)
- Hosting/Datacenter flag — cloud provider or VPS (not residential)
- Direct Google Maps link with pin at coordinates
- Direct Shodan link for the IP
- Direct GreyNoise link (known scanner/malicious activity)
- Direct AbuseIPDB link (reported abuse history)

7.3 IP Classification and Action

RESIDENTIAL — Not proxy/hosting	Submit legal process to ISP for subscriber name, address, ID. Use Section 91 CrPC or IT Act Section 69.
DATACENTER / HOSTING	Submit abuse request to hosting provider. Request account records, payment info, email used to register.

VPN / PROXY DETECTED	Known proxy exit. Correlate with other artifacts for real IP. Request VPN provider logs via MLAT if foreign.
Tor Exit Node	Expected — Tor fetch exits through Tor. Other IPs in page source are more valuable targets.

7.4 ISP Legal Process for India

- BSL/BSNL, Reliance Jio, Airtel, Vodafone-Idea, MTNL: Contact via written request to their Nodal Officer under IT Act Section 69.
- Include: IP address, date/time with timezone, protocol (HTTP/Tor), purpose of request.
- Foreign ISPs: Submit via Mutual Legal Assistance Treaty (MLAT) or Interpol NCB channel.
- Foreign Hosting (AWS, DO, Vultr etc): Submit to their Law Enforcement portal or abuse team.

8. READING THE ATTRIBUTION GRAPH

The attribution graph connects every extracted artifact into a visual evidence network. It is the core deliverable for briefing supervisors and prosecutors.

8.1 Node Types

🕸 onion_site (blue hexagon)	The target .onion URL — root node of all evidence
@ email (red circle)	Email address found in page source
✈ telegram (light blue)	Telegram handle or group
₿ btc_address (orange)	Bitcoin wallet address
Ξ eth_address (indigo)	Ethereum wallet address
⊕ ip_address (red — critical)	IP address — geolocation available
📍 location (teal)	Physical location derived from IP geolocation
🌐 isp (grey)	ISP / hosting provider — legal process target
👤 github_profile (white)	GitHub account correlated to artifact
👤 reddit_profile (orange-red)	Reddit account correlated to artifact
🔑 pgp_key (green)	PGP key found or linked
★ analytics_id (pink — critical)	Google Analytics / Facebook Pixel ID
\$ exchange (green — critical)	Known crypto exchange — KYC legal process
🌐 domain (yellow)	Cleartext domain linked from .onion site

8.2 Edge Types (Evidence Chains)

real_ip_leak	CRITICAL — X-Forwarded-For header exposed operator real IP
registered_with	CRITICAL — account or PGP key registered with this email (real identity)
same_identity	HIGH — username found on GitHub/Reddit under same handle
deposited_to	CRITICAL — BTC address sent funds to known exchange
located_in	HIGH — IP geolocated to this city/country
tracked_by	CRITICAL — site uses this analytics ID (links to cleartext site)
payment_via	HIGH — site accepts this crypto address
signed_with	MEDIUM — site content signed with this PGP key

8.3 Interacting With the Graph

- Drag nodes to rearrange the layout
- Scroll to zoom in/out
- Hover over any node for full details panel (type, risk level, all data)
- CRITICAL nodes (red) and CRITICAL edges glow — these are your priority leads
- 'High-Value Evidence Chains' panel below graph lists the most important paths

9. IDENTITY CORRELATION ENGINE

Real investigations succeed by finding the same person across multiple platforms. UMBRA automatically searches every username, email, and PGP key it finds against live internet databases.

9.1 GitHub API

- Searches username directly: `api.github.com/users/{username}`
- If account has public email set: **CRITICAL** — this is the operator's real email address
- Location, company, bio, creation date — all feed into operator profile
- 60 requests/hour unauthenticated. Create PAT at `github.com/settings/tokens` for higher limits.

9.2 Reddit API

- Checks: `reddit.com/user/{username}/about.json`
- Returns: karma, account creation date, moderator status
- Account age + karma builds operator timeline. Check post/comment history manually.

9.3 OpenPGP Keyserver

⚠ PGP keys are the strongest identity link. Operators who sign their content and upload their PGP key to a keyserver permanently attach their real email to their alias. Many criminals make this mistake.

- Searches `keys.openpgp.org` for email or username
- If key found: extracts `email_in_key` — this is the operator's real registered email
- Fingerprint, creation date, algorithm — all logged for further investigation

9.4 Google Dorks (Manual Queries)

UMBRA generates ready-to-run Google dork queries for each artifact. Click 'SEARCH →' to open in browser:

- "username" — direct string search across all indexed pages
- "username" site:reddit.com — Reddit mentions
- "username" site:github.com — GitHub repositories and profiles
- "username" site:pastebin.com — paste data, often credentials or contact info
- "username" email OR contact — may find operator email elsewhere
- "btc_address" site:bitcointalk.org — forum posts using same wallet

9.5 Wallet Explorer (Bitcoin Exchange Labels)

- Checks WalletExplorer.com for known exchange labels on BTC addresses
- If exchange detected (Binance, Kraken, Coinbase etc): submit legal process for KYC
- Exchange KYC records include: verified name, national ID, address, phone number

10. INFRASTRUCTURE FINGERPRINTING

10.1 Analytics ID Reuse — Most Powerful Technique

⚠ Analytics IDs are the #1 historical method for exposing dark web operators. The same Google Analytics ID used on an onion site and a clearnet site directly links the operator to their real identity.

- Google Analytics UA-XXXXXXXX-X and GA4 G-XXXXXXXXXX
- Facebook Pixel IDs — can reveal ad account and Facebook identity
- Stripe publishable keys — legal process to Stripe for merchant identity
- AWS S3 bucket names — can reveal account owner via abuse request to AWS

Steps when analytics ID found:

6. Copy the ID (e.g. UA-1234567-1)
7. Search Google: "UA-1234567-1" — finds clearnet sites using same property
8. Run Shodan: `http.html:"UA-1234567-1"` — finds servers with this code
9. Use DomainTools or BuiltWith.com to find all domains using this ID

10.2 Favicon Hash (Shodan Correlation)

- Downloads favicon.ico from the .onion site
- Computes MurmurHash3 (exact Shodan algorithm — mmh3 library)
- Generates exact Shodan query: `http.favicon.hash:{hash}`
- Run on Shodan to find ALL servers (clearnet + onion) using this exact icon
- Often reveals the clearnet server running the same template or CMS

10.3 JavaScript Fingerprint

- Hashes all script src= URLs using SHA-256
- Same fingerprint across multiple onion sites = same operator or template provider
- External script sources (clearnet CDN URLs) are logged separately — CRITICAL
- A clearnet script URL loaded by an onion site may reveal the hosting infrastructure

11. TROUBLESHOOTING — Common Issues & Fixes

11.1 Tor Issues

Tor timeout on .onion fetch	Run: <code>sudo systemctl restart tor && sleep 20</code> . Try again. Some .onion sites are slow or offline.
IsTor:false in check	Tor routing broken. Run: <code>sudo systemctl restart tor</code> . Wait 30 seconds. Retry check.
Connection refused :9050	Tor not started. Run: <code>sudo systemctl start tor</code>
New circuit not working	ControlPort not configured. Re-check <code>/etc/tor/torrc</code> <code>HashedControlPassword</code> line.
Site returns 400/403/503	Normal — some sites have bot detection. Check if site is online via Tor Browser manually.

11.2 Backend Issues

Port 8000 already in use	Run: <code>kill \$(lsof -ti:8000)</code> then restart backend
ImportError: No module named X	Run: <code>source venv/bin/activate && pip install X</code>
lxml installation fails	Run: <code>sudo apt install libxml2-dev libxslt-dev python3-dev && pip install lxml --force-reinstall</code>
mmh3 fails to install	Run: <code>sudo apt install gcc python3-dev && pip install mmh3</code>
requests[socks] error	Run: <code>pip install "requests[socks]" PySocks</code>
anthropic module error	Run: <code>pip install anthropic --upgrade</code>

11.3 API Issues

ip-api.com returns fail	Private or reserved IP address — filtered out automatically by UMBRA
ip-api.com rate limit	Free limit: 45 req/min. UMBRA waits 0.4s between calls. Should not trigger.
GitHub returns 403	Rate limited (60/hr anon). Set <code>GITHUB_TOKEN</code> env var or wait an hour.
Blockchair no data	BTC address not indexed or new address with 0 transactions. Manual link shown.
WalletExplorer no label	Address not in WalletExplorer database. Not all addresses are labeled.
Claude API error	Check <code>ANTHROPIC_API_KEY</code> is set correctly. Verify at <code>console.anthropic.com</code> .

11.4 Frontend Issues

CORS error in browser console	Backend must run with CORS enabled (already configured in <code>umbra v3_backend.py</code>)
UI shows no data after scan	Open browser console (F12). Check for network errors. Verify backend is running.
D3 graph not rendering	D3 import issue. Use Claude.ai artifact renderer or ensure 'd3' npm package installed.

Timeout waiting for results	Normal for slow .onion sites. Wait up to 120 seconds. Site may be offline.
------------------------------------	--

12. SAFE SHUTDOWN PROCEDURE

⚠ Follow this procedure after every operational session to ensure no sensitive data remains in memory or temp files.

STEP 01 Stop the Backend

In the terminal running `umbra_v3_backend.py`:

```
| Ctrl + C
```

Verify it stopped: the prompt returns and no more log output appears.

STEP 02 Close the Frontend

Close the browser tab/window running the UMBRA UI. If using React dev server:

```
| Ctrl + C # in the terminal running npm start
```

STEP 03 Deactivate Virtual Environment

```
| deactivate
```

Your prompt should no longer show `'(venv)'`.

STEP 04 Stop Tor (Optional — for OPSEC)

If you do not need Tor running after your session:

```
| sudo systemctl stop tor
```

STEP 05 Clear Any Sensitive Outputs (If Required)

UMBRA does not write intelligence reports to disk by default — all data is in browser memory and cleared when tab closes. If you exported any reports:

```
| shred -vzu -n 3 /path/to/exported/report.txt
```

For secure temporary file removal:

```
| sudo srm -rf /tmp/umbra_* # if srm is installed
```

STEP 06 Session Complete — Verification

Verify clean shutdown:

```
| # Verify backend stopped:
| curl http://localhost:8000/ # Should return: connection refused
| # Verify Tor stopped (if you stopped it):
| systemctl is-active tor # Should return: inactive
```

- ✓ Backend stopped and not accepting connections
- ✓ Browser tab closed — no data in browser memory
- ✓ Virtual environment deactivated
- ✓ Tor stopped (if operational security requires it)

13. LIVE DEMONSTRATION SCRIPT — NIA / LEA

Use this exact sequence for a live demonstration in front of agency personnel. Total time: approximately 15–20 minutes.

⚠ Test with a known .onion site BEFORE the demonstration. Have a backup target ready in case the primary site is offline.

Demonstration Flow

00:00 — Setup Check (2 min)	Run <code>verify.sh</code> . Show green checkmarks. Explain: Tor active, backend running, all APIs ready.
02:00 — Problem Statement (2 min)	Show the .onion URL to audience. Explain: operator is behind Tor + VPN. Our goal: extract intelligence to support attribution.
04:00 — Run Analysis (2 min)	Paste target URL. Click Analyze. Show the live progress log as modules run. Narrate each step.
06:00 — IP Intelligence (3 min)	Click IP tab. Show geolocation data. Explain residential vs datacenter vs VPN. Show Google Maps pin. Explain ISP legal process.
09:00 — Attribution Graph (3 min)	Click Graph tab. Walk through nodes. Show high-value evidence chains. Explain how each link was found.
12:00 — AI OSINT Brief (3 min)	Show Claude brief in Results tab. Walk through operator profile and top 5 leads. Emphasize copy-paste ready queries.
15:00 — Identity Correlation (2 min)	Show GitHub/Reddit/PGP findings if present. Demonstrate Google dorks. Explain cross-platform identity linking.
17:00 — Questions	Discuss legal process, MLAT, further investigation steps.

Key Talking Points

- All intelligence is extracted from what the operator inadvertently exposes — not from attacking the Tor network
- Every finding is automatically linked to the next investigation step (Shodan query, legal process, keyserver search)
- The attribution graph shows judges and prosecutors the evidence chain in a visual format they can understand
- Operator OPSEC failures are the primary attack surface — analytics IDs, IP leaks, username reuse, PGP email registration
- This platform is deployable today — no further setup needed for operational use

14. FILES REFERENCE

Deliverable Files

umbra_v3_backend.py	1128-line Python FastAPI backend. All 12 intelligence engines. Run: <code>python3 umbra_v3_backend.py</code>
umbra_v3_frontend.jsx	778-line React + D3 frontend. 7-tab UI. Load in Claude.ai or React dev server.
UMBRA_V3_GUIDE.docx	This document. Complete deployment and operations guide.

Backend API Endpoints

GET /	Health check. Returns version and status.
GET /api/tor/check	Verifies Tor connection. Returns exit IP + is_tor flag.
GET /api/tor/newcircuit	Requests new Tor circuit via stem/ControlPort.
POST /api/analyze	Main endpoint. Full 12-module analysis. Body: {onion_url, anthropic_api_key}
GET /api/geo/{ip}	Standalone IP geolocation + VPN detection.
GET /api/ct/{domain}	Standalone Certificate Transparency lookup.
GET /api/btc/{address}	Standalone blockchain forensics lookup.
GET /docs	Interactive Swagger API documentation.

Python Dependencies

fastapi + uvicorn	Web framework and ASGI server for the REST API
requests[socks]	HTTP client with SOCKS5 proxy support for Tor
PySocks	SOCKS5 library required by requests
stem	Tor control protocol library (new circuit requests)
anthropic	Anthropic Claude API client (OSINT brief generation)
beautifulsoup4	HTML parsing for page intelligence extraction
lxml	Fast XML/HTML parser backend for BeautifulSoup
pydantic	Request/response data validation
mmh3	MurmurHash3 for Shodan-compatible favicon hashing

External APIs Used (All Free)

check.torproject.org	Tor exit verification – confirms traffic routing through Tor
ip-api.com	IP geolocation: country, city, lat/lon, ISP, VPN detection. 45 req/min free
blockchair.com	Bitcoin blockchain: balance, transactions, activity timeline
walletexplorer.com	Bitcoin wallet exchange labels for KYC identification

api.github.com	GitHub user profiles and code search. 60 req/hr unauthenticated
reddit.com/user/.../about.json	Reddit account data. Unauthenticated.
keys.openpgp.org	OpenPGP keyserver search. Real email extraction from PGP keys.
crt.sh	Certificate Transparency log search. Full cert history for clearnet domains.
api.anthropic.com	Claude Opus AI — NIA OSINT brief generation. Requires API key.

15. ANALYST OPSEC — Protecting the Investigation

⚠ Operational security protects both the analyst and the investigation. Follow these rules during all intelligence collection sessions.

Network OPSEC

- Run UMBRA backend on a dedicated analysis machine — not your daily-use laptop
- Use a wired connection when possible — Wi-Fi can be monitored
- All .onion fetches go through Tor automatically — your real IP is not exposed to target sites
- The ip-api.com, GitHub, Reddit, crt.sh, Blockchair API calls go over clearnet from your machine — this is expected and only reveals that someone queried these APIs, not what the query was about
- For maximum OPSEC: run the backend on a Tails OS or air-gapped machine, exfiltrate results via encrypted USB

Data Handling

- UMBRA does not log or store intelligence reports to disk — all data lives in browser memory
- When session ends and browser tab is closed, all extracted intelligence is gone
- If you need to preserve a report: export/screenshot within the session, encrypt before storing
- Never copy .onion URLs or operator PII into unsecured messaging apps or email

Investigation OPSEC

- Do not query the same .onion site repeatedly in short succession — operators may notice unusual traffic patterns
- Request new Tor circuit (NEW CIRCUIT button) between separate investigations
- Google dork results must be opened in an incognito window or Tor Browser — not your regular Chrome profile
- Do not access GitHub/Reddit profiles from your personal account after investigation — use anonymous browser session

Legal Authority

- UMBRA collects only publicly available information — no hacking, no exploitation, no unauthorized access
- All intelligence is derived from: public HTTP responses, public APIs, public blockchain records, public certificate logs

- Admissibility: document your methodology at each step for chain of custody
- Indian legal authority: IT Act 2000/2008 Section 69, UAPA, NDPS Act, Arms Act — consult your legal team for applicable sections per case